

✓ Actividad Guiada 1 de Algoritmos de Optimización

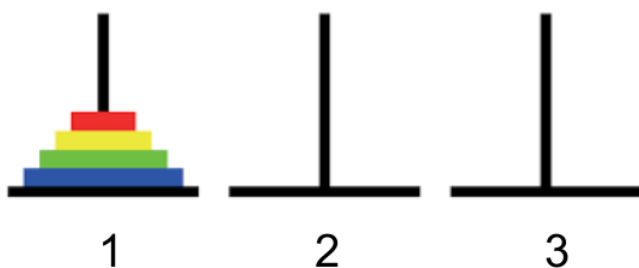
Nombre: David Villacorta Nicolás

Cuaderno: <https://colab.research.google.com/drive/19TyeGzpPc8FXbQHTuCZo0Bvt44c0dNru>

Proyecto GitHub: <https://github.com/davidvilla10/O3MIAR-Algoritmos-de-Optimizacion>

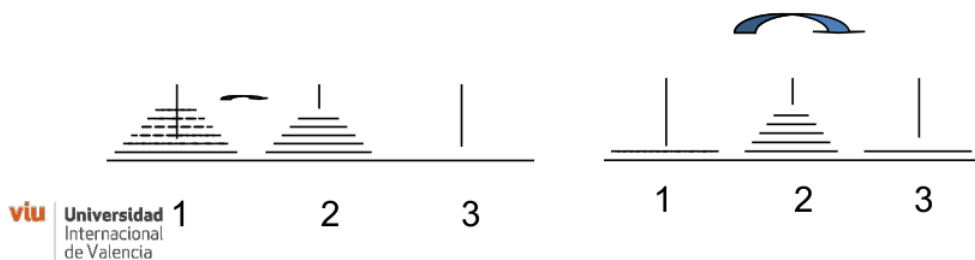
```
# Imports necesarios
import random
```

✓ 1.Torres de Hanoi - Divide y vencerás



Resolver(Total_fichas=4, Desde=1, Hasta=3) es valido con:

- Resolver(Total_fichas=3, Desde=1, Hasta=2)
- Mover(Desde=1, Hasta=3)
- Resolver(Total_fichas=3, Desde=2, Hasta=3)



```
#Torres de Hanoi - Divide y venceras
#####
'''
La explicación general de la función Resolver_Torres_Hanoi() es la siguiente:
dividimos el problema en subproblemas más pequeños aplicando la estrategia
de "divide y vencerás".

Para mover N fichas desde la torre origen hasta la torre destino:
1. Primero movemos las N-1 fichas superiores desde la torre origen hasta la
   torre auxiliar (la torre que no es ni origen ni destino).
2. Luego movemos la ficha más grande directamente desde la torre
   origen hasta la torre destino.
3. Finalmente, movemos las N-1 fichas desde la torre auxiliar hasta la torre
   destino, utilizando de nuevo la torre origen como auxiliar.

Repetimos el proceso recursivamente hasta llegar al caso base, en el que
solo hay una ficha, que se puede mover directamente de la torre origen a la
torre destino.
'''

#####
def Resolver_Torres_Hanoi(N, desde, hasta):
    #N - Nº de fichas
    #desde - torre inicial
    #hasta - torre final
    if N==1 :
```

```

        print("Lleva la ficha desde " + str(desde) + " hasta " + str(hasta))

    else:
        Resolver_Torres_Hanoi(N-1, desde, 6-desde-hasta)
        print("Lleva la ficha desde " + str(desde) + " hasta " + str(hasta))
        Resolver_Torres_Hanoi(N-1, 6-desde-hasta, hasta)

Resolver_Torres_Hanoi(5, 1, 3)
#####

```

```

Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 3 hasta 1
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 3 hasta 1
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 1 hasta 2
Lleva la ficha desde 3 hasta 2
Lleva la ficha desde 1 hasta 3
Lleva la ficha desde 2 hasta 1
Lleva la ficha desde 2 hasta 3
Lleva la ficha desde 1 hasta 3

```

1.1 Complejidad del algoritmo de Torres de Hanoi

Sea $T(n)$ el número de operaciones necesarias para mover n fichas.

Caso base

Para $n = 1$, el algoritmo realiza un único movimiento: $T(1) = 1$

Caso recursivo

Para $n > 1$, el algoritmo realiza:

1. Una llamada recursiva para mover $n - 1$ fichas a la torre auxiliar.
2. Un movimiento de la ficha mayor.
3. Otra llamada recursiva para mover $n - 1$ fichas a la torre destino.

Por tanto, la relación de recurrencia es $T(n) = 2 \cdot T(n - 1) + 1$

Resolución de la recurrencia

Desarrollando la recurrencia:

$$\begin{aligned}
 T(n) &= 1 + 2 \cdot T(n - 1) \\
 &= 1 + 2(1 + 2 \cdot T(n - 2)) \\
 &= 1 + 2 + 2^2 + \dots + 2^{n-1}T(1)
 \end{aligned}$$

Sustituyendo $T(1) = 1$, tenemos $T(n) = 1 + 2 + 2^2 + \dots + 2^{n-1}$

Esta suma es una serie geométrica y, aplicando la fórmula $a + ar + ar^2 + \dots + ar^n = a \left(\frac{1-r^{n+1}}{1-r} \right)$, se obtiene $T(n) = 2^n - 1$

Complejidad asintótica

Dado que el término dominante es 2^n , la complejidad del algoritmo es: $O(2^n)$

2. Cambio de monedas - Técnica voraz

```

#Cambio de monedas - Técnica voraz
...

Dado un sistema de monedas ordenado decrecientemente, el algoritmo selecciona
en cada paso la moneda de mayor valor posible que no supere la cantidad
restante por devolver. Es decir, en cada iteración tomamos la decisión local
óptima (aquí entra en juego la técnica voraz), con la esperanza de que
esta elección conduzca a una solución global óptima.

El proceso consiste en:

```

1. Inicializamos una solución con cero monedas de cada tipo.
2. Recorremos el sistema de monedas de mayor a menor valor.
3. Para cada moneda, calculamos cuántas veces se puede utilizar sin superar la cantidad pendiente por devolver.
4. Actualizamos la cantidad total devuelta.
5. Finalizamos cuando la cantidad devuelta coincide con la cantidad objetivo.

Si al finalizar el recorrido no se alcanza exactamente la cantidad deseada, no es posible encontrar una solución con el sistema de monedas dado.

```
#####
def cambio_monedas(CANTIDAD,SISTEMA):

    SOLUCION = [0]*len(SISTEMA)
    ValorAcumulado = 0

    for i,valor in enumerate(SISTEMA):
        monedas = (CANTIDAD-ValorAcumulado)//valor
        SOLUCION[i] = monedas
        ValorAcumulado = ValorAcumulado + monedas*valor

    if CANTIDAD == ValorAcumulado:
        return SOLUCION

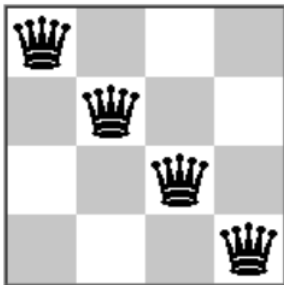
    print("No es posible encontrar solucion")

SISTEMA = [11, 5, 1]
print(f"Para el sistema: {SISTEMA}, la solución voraz es: {cambio_monedas(15,SISTEMA)}")
SISTEMA = [25, 10, 5, 1]
print(f"Para el sistema: {SISTEMA}, la solución voraz es: {cambio_monedas(27,SISTEMA)}")

#####
```

```
Para el sistema: [11, 5, 1], la solución voraz es: [1, 0, 4]
Para el sistema: [25, 10, 5, 1], la solución voraz es: [1, 0, 0, 2]
```

3.N Reinas - Vuelta Atrás(Backtracking)



```
#N Reinas - Vuelta Atrás()
#####
...

La solución del problema de N Reinas con Vuelta Atrás se compone de varias
funciones.

1. En primer lugar, es_prometedora(), que verifica que en una solución
parcial no hay amenazas entre reinas (es decir, no hay dos reinas en la misma
columna, fila o diagonal).
2. En segundo lugar, reinas(), que explora todas las posibilidades del espacio
de soluciones. Recorre iterativamente todas las columnas y para cada columna
prueba a colocar una reina en cada fila y llamarse recursivamente para
probar todas las combinaciones de reinas en las filas siguientes a la de la
iteración actual. Es decir, explora todas las posibles soluciones y detecta
cuales son soluciones finales del problema.
...

#Verifica que en la solución parcial no hay amenazas entre reinas
#####
def es_prometedora(SOLUCION,etapa):
    #####
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos reinas en la misma fila
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION.count(SOLUCION[i])) + " veces")
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

    #Verifica las diagonales
    for j in range(i+1, etapa +1 ):
        #print("Comprobando diagonal de " + str(i) + " y " + str(j))
        if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
```

```

return True

#Traduce la solución al tablero
#####
def escribe_solucion(S):
#####
    n = len(S)
    for x in range(n):
        print("")
        for i in range(n):
            if S[i] == x+1:
                print(" X ", end="")
            else:
                print(" - ", end="")

#Proceso principal de N-Reinas
#####
def reinas(N, solucion=[],etapa=0):
#####
    if len(solucion) == 0:
        # [0,0,0...]
        solucion = [0 for i in range(N) ]

    for i in range(1, N+1):
        solucion[etapa] = i
        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

    solucion[etapa] = 0

reinas(5,solucion=[],etapa=0)

```

```

[1, 3, 5, 2, 4]
[1, 4, 2, 5, 3]
[2, 4, 1, 3, 5]
[2, 5, 3, 1, 4]
[3, 1, 4, 2, 5]
[3, 5, 2, 4, 1]
[4, 1, 3, 5, 2]
[4, 2, 5, 3, 1]
[5, 2, 4, 1, 3]
[5, 3, 1, 4, 2]

```

```

escribe_solucion([1, 5, 8, 6, 3, 7, 2, 4])

```

```

X - - - - -
- - - - - X -
- - - - X - -
- - - - - - X
- X - - - - -
- - - X - - -
- - - - X - -
- - X - - - -

```

4.Práctica Individual: Dos puntos más cercanos

4.1 Caso 1D: Fuerza Bruta

```

#####
def cercanos_1D_fuerza_bruta(lista):
    '''
    Encuentra los dos puntos más cercanos en una lista de números
    usando fuerza bruta. Se comparan todas las parejas posibles y se
    devuelve la mínima distancia junto con los dos puntos correspondientes.
    '''
    n = len(lista)
    min_distancia = float('inf') # Usamos infinito para inicializar la distancia mínima y así actualizar en la primera comparación
    par = (None, None) # En esta tupla devolveremos los dos puntos más cercanos

    # Con un bucle doble recorremos por fuerza bruta todas las posibilidades
    for i in range(0, n):
        for j in range(i+1, n):
            distancia = abs(lista[i] - lista[j])
            if distancia < min_distancia:
                min_distancia = distancia
                par = (lista[i], lista[j])

    return par, min_distancia

#####

```

```

LISTA_1D = [random.randrange(1, 10000) for x in range(1000)] # Generamos una lista con 1000 valores aleatorios
LISTA_1D_sin_repeticiones = random.sample(range(1, 10001), 1000) # Es una lista como la anterior pero sin repetidos

par, distancia = cercanos_1D_fuerza_bruta(LISTA_1D)
print(f"Los dos puntos más cercanos de la lista con posibles repeticiones son: {par} con distancia {distancia}")
par, distancia = cercanos_1D_fuerza_bruta(LISTA_1D_sin_repeticiones)
print(f"Los dos puntos más cercanos de la lista sin posibles repeticiones son: {par} con distancia {distancia}")

Los dos puntos más cercanos de la lista con posibles repeticiones son: (1694, 1694) con distancia 0
Los dos puntos más cercanos de la lista sin posibles repeticiones son: (2225, 2224) con distancia 1

```

4.1.1 Complejidad del algoritmo de puntos más cercanos en 1D (fuerza bruta)

Sea $T(n)$ el número de operaciones necesarias para encontrar los dos puntos más cercanos en una lista de n números.

Caso base

Para $n = 2$, solo hay un par posible, por lo que el algoritmo realiza un único cálculo de distancia:

$$T(2) = 1$$

Caso general

Para $n > 2$, el algoritmo compara todas las parejas posibles de puntos:

1. Para el primer punto, se comparan con los $n - 1$ puntos restantes.
2. Para el segundo punto, se comparan con los $n - 2$ puntos restantes.
3. Y así sucesivamente hasta que todos los pares se hayan evaluado.

Por tanto, el número total de comparaciones es la suma de los primeros $n - 1$ enteros positivos:

$$T(n) = 1 + 2 + 3 + \dots + (n - 1)$$

Resolución de la recurrencia / fórmula cerrada

La suma anterior es una serie aritmética, que se puede calcular mediante:

$$T(n) = \frac{(n-1) \cdot n}{2}$$

Complejidad asintótica

Dado que el término dominante es $n^2/2$, la complejidad del algoritmo es: $O(n^2)$.

4.2 Caso 1D: Divide y Vencerás

```

#####
def cercanos_1D_divide_y_venceras(lista):
    ...

    Encuentra los dos puntos más cercanos en una lista 1D usando
    Divide y Vencerás. Primero ordena la lista y luego divide el
    problema en sublistas, resolviendo recursivamente y combinando
    las soluciones comparando únicamente los puntos vecinos.

    NOTA: Es importante que al final, una vez se tiene la menor
    distancia entre la lista derecha y la izquierda, se compara
    la distancia entre los puntos de la frontera (o sea el último
    punto de la lista izquierda con el primero de la lista derecha).
    Si no hiciéramos esto, estaríamos perdiendo una posible pareja
    óptima dentro de nuestra lista.
    ...

    lista_ordenada = sorted(lista)

    def resolver(lista):
        n = len(lista)

        if n < 2: # No hay dos puntos, solo uno. Devolvemos 'inf' para que no sea considerado entre las distancias óptimas
            return None, float('inf')
        if n == 2:
            return (lista[0], lista[1]), abs(lista[0] - lista[1])

        mitad = n // 2
        izquierda = lista[:mitad]
        derecha = lista[mitad:]

        (p1, d1) = resolver(izquierda)
        (p2, d2) = resolver(derecha)

        if d1 < d2:
            mejor_par, menor_distancia = p1, d1
        else:
            mejor_par, menor_distancia = p2, d2

        # Comprobamos si los puntos frontera están a menor distancia que la solución actual
        frontera_distancia = abs(izquierda[-1] - derecha[0])
        if frontera_distancia < menor_distancia:

```

```

mejor_par = (izquierda[-1], derecha[0])
menor_distancia = frontera_distancia

return mejor_par, menor_distancia

return resolver(lista_ordenada)

#####

```

```

LISTA_1D = [random.randrange(1, 10000) for x in range(1000)] # Generamos una lista con 1000 valores aleatorios
LISTA_1D_sin_repeticiones = random.sample(range(1, 10001), 1000) # Es una lista como la anterior pero sin repetidos

par, distancia = cercanos_1D_divide_y_venceras(LISTA_1D)
print(f"Los dos puntos más cercanos de la lista con posibles repeticiones son: {par} con distancia {distancia}")
par, distancia = cercanos_1D_divide_y_venceras(LISTA_1D_sin_repeticiones)
print(f"Los dos puntos más cercanos de la lista sin posibles repeticiones son: {par} con distancia {distancia}")

Los dos puntos más cercanos de la lista con posibles repeticiones son: (9888, 9888) con distancia 0
Los dos puntos más cercanos de la lista sin posibles repeticiones son: (9938, 9939) con distancia 1

```

4.2.1 Complejidad del algoritmo de puntos más cercanos en 1D (Divide y Vencerás)

Sea $T(n)$ el número de operaciones necesarias para encontrar los dos puntos más cercanos en una lista de n números.

Caso base

- Para $n = 2$, se calcula una única distancia entre los dos puntos:
 $T(2) = 1$

Caso recursivo

Para $n > 2$, el algoritmo realiza:

- Una división de la lista ordenada en dos mitades de tamaño aproximado $n/2$.
- Una llamada recursiva para encontrar los puntos más cercanos en la mitad izquierda.
- Una llamada recursiva para encontrar los puntos más cercanos en la mitad derecha.
- Algunas operaciones elementales que no dependen de n , como la comparación para elegir el mejor par, una comparación adicional entre los puntos frontera, asignaciones y returns.

Por tanto, la relación de recurrencia es:

$T(n) = 2 \cdot T(n/2) + c$, donde c representa todas esas operaciones elementales en cada llamada.

Resolución de la recurrencia

En la diapositiva 27/41 del pdf de la Actividad Guiada 1 aparecen las diferentes soluciones a una ecuación en recurrencia de la forma $a \cdot T(\frac{n}{b}) + c \cdot n^k$. Nos encontramos en el caso $a = 2$, $b = 2$, y $k = 0$ luego tenemos que $a = 2 > b^k = 1$ y por tanto

$$T(n) = O(n^{\log_b a}) = O(n^{\log_2 2}) = O(n)$$

Sin embargo, el algoritmo requiere ordenar previamente la lista, lo cual tiene un coste adicional: $O(n \log n)$

Complejidad asintótica

Sumando el coste de la ordenación y el proceso recursivo, la complejidad total del algoritmo es: $O(n \log n)$

¿Se puede mejorar la fuerza bruta?

Acabamos de demostrar que sí se puede encontrar un algoritmo mejor que el de fuerza bruta (que tenía complejidad $O(n^2)$). Sin embargo, no podemos bajar de la complejidad $O(n \log n)$. Esto se debe a que, para resolver este problema, la mejor estrategia es ordenar la lista de puntos iniciales, pues si no nos veríamos obligados a hacer más comparaciones de las necesarias. Ordenando la lista ponemos el foco en dónde podrían estar los puntos más cercanos. Pero como hemos visto en clase, el mínimo coste para ordenar una lista es $O(n \log n)$, por lo que aunque ordenásemos la lista de la manera más eficiente posible, no podríamos bajar de ese orden de complejidad.

4.3 Caso 2D: Divide y Vencerás

Vamos a generalizar el algoritmo aplicado en el punto 5.2 para el caso 2D.

```

LISTA_2D = [(random.randrange(1,10000), random.randrange(1,10000)) for x in range(1000)]

#####
def cercanos_2D_divide_y_venceras(lista):
    ...
    Encuentra los dos puntos más cercanos en un conjunto de puntos en 2D
    usando Divide y Vencerás. El algoritmo ordena los puntos por coordenada
    x, divide el conjunto en dos mitades, resuelve recursivamente y combina
    comparando los puntos cercanos a la frontera central.

    Nota 1: En la fase de combinación solo se comparan puntos cuya diferencia en x
    es menor que la mejor distancia encontrada, lo que garantiza eficiencia.
    Nota 2: En el caso 1D la frontera eran solo dos puntos. En el caso de 2D, la frontera

```

```

es una línea de puntos que hay que recorrer.
Nota 3: En 2D, se considera la distancia euclídea.
'''

puntos_x = sorted(lista, key=lambda p: p[0])

def distancia(p1, p2):
    return ((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2) ** 0.5

def resolver(puntos):
    n = len(puntos)

    if n < 2:
        return None, float('inf')

    if n == 2:
        return (puntos[0], puntos[1]), distancia(puntos[0], puntos[1])

    mitad = n // 2
    izquierda = puntos[:mitad]
    derecha = puntos[mitad:]

    (par_i, dist_i) = resolver(izquierda)
    (par_d, dist_d) = resolver(derecha)

    if dist_i < dist_d:
        mejor_par, mejor_dist = par_i, dist_i
    else:
        mejor_par, mejor_dist = par_d, dist_d

    x_medio = puntos[mitad][0]
    franja = [p for p in puntos if abs(p[0] - x_medio) < mejor_dist]
    franja.sort(key=lambda p: p[1])

    for i in range(len(franja)):
        for j in range(i + 1, min(i + 7, len(franja))):
            d = distancia(franja[i], franja[j])
            if d < mejor_dist:
                mejor_dist = d
                mejor_par = (franja[i], franja[j])

    return mejor_par, mejor_dist

return resolver(puntos_x)

#####

```

```

LISTA_2D = [(random.randrange(1, 10000), random.randrange(1, 10000)) for x in range(1000)]
par, distancia = cercanos_2D_divide_y_venceras(LISTA_2D)
print(f"Los dos puntos más cercanos son: {par} con distancia {distancia}")

```

Los dos puntos más cercanos son: ((4216, 5727), (4217, 5730)) con distancia 3.1622776601683795

4.3.1 Complejidad del algoritmo de puntos más cercanos en 2D (Divide y Vencerás)

Sea $T(n)$ el número de operaciones necesarias para encontrar los dos puntos más cercanos en un conjunto de n puntos en el plano.

Caso base

- Para $n = 2$, se calcula una única distancia entre los dos puntos:
 $T(2) = 1$

Caso recursivo

Para $n > 2$, el algoritmo realiza:

- Una división del conjunto de puntos ordenado por la coordenada x en dos mitades de tamaño aproximado $n/2$.
- Una llamada recursiva para encontrar los puntos más cercanos en la mitad izquierda.
- Una llamada recursiva para encontrar los puntos más cercanos en la mitad derecha.
- Un proceso de combinación que analiza los puntos cercanos a la frontera central, construyendo una franja de ancho igual a la mejor distancia encontrada y comparando un número constante de puntos para cada uno de ellos.

Por tanto, la relación de recurrencia es:

$$T(n) = 2 \cdot T(n/2) + O(n)$$

donde el término $O(n)$ corresponde al coste de la fase de combinación. Puede parecer que tenemos un doble bucle (lo que nos llevaría a $O(n^2)$) pero en realidad el bucle de la j no crece con el tamaño de entrada dado que la j solo recorrerá un número constante de puntos (máximo 6 para cada i porque por geometría solo puede haber 6 puntos dentro de ese área que estén separados en una distancia d). Por ese motivo ordenar los puntos de la franja con sort tampoco nos aumenta el término de $O(n)$ a $O(n \log n)$.

Resolución de la recurrencia

En la diapositiva 27/41 del pdf de la Actividad Guiada 1 aparecen las diferentes soluciones a una ecuación en recurrencia de la forma

$a \cdot T\left(\frac{n}{b}\right) + c \cdot n^k$. En este caso tenemos $a = 2$, $b = 2$, $f(n) = n$ y $k = 1$, por lo que:

$$b^k = 2^1 = 2 = a$$

Por tanto se cumple que:

$$T(n) = O(n^k \log n) = O(n \log n)$$

Complejidad asintótica

El algoritmo requiere ordenar inicialmente los puntos por la coordenada x , lo cual tiene un coste $O(n \log n)$. Este coste es del mismo orden que el de la resolución recursiva, por lo que la complejidad temporal total del algoritmo es: $O(n \log n)$

4.4 Caso 3D: Divide y Vencerás

Vamos a generalizar el algoritmo aplicado en el punto 5.2 para el caso 3D

```
#####
def cercanos_3D_divide_y_vencerás(lista):
    '''
    Encuentra los dos puntos más cercanos en un conjunto de puntos en 3D
    usando Divide y Vencerás. El algoritmo ordena los puntos por coordenada x,
    divide el conjunto en dos mitades, resuelve recursivamente y combina
    comparando los puntos cercanos a la frontera central.

    Nota 1: En la fase de combinación solo se comparan puntos cuya diferencia en x
    es menor que la mejor distancia encontrada, lo que garantiza eficiencia.
    Nota 2: En 3D, la distancia euclídea se calcula con tres coordenadas.
    '''

    puntos_x = sorted(lista, key=lambda p: p[0])

    def distancia(p1, p2):
        return ((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2 + (p1[2] - p2[2])**2) ** 0.5

    def resolver(puntos):
        n = len(puntos)

        if n < 2:
            return None, float('inf')

        if n == 2:
            return (puntos[0], puntos[1]), distancia(puntos[0], puntos[1])

        mitad = n // 2
        izquierda = puntos[:mitad]
        derecha = puntos[mitad:]

        par_i, dist_i = resolver(izquierda)
        par_d, dist_d = resolver(derecha)

        if dist_i < dist_d:
            mejor_par, mejor_dist = par_i, dist_i
        else:
            mejor_par, mejor_dist = par_d, dist_d

        # Generalizamos el caso de 1D a este caso de 2D
        x_medio = puntos[mitad][0]
        franja = [p for p in puntos if abs(p[0] - x_medio) < mejor_dist]

        franja.sort(key=lambda p: (p[1], p[2]))

        for i in range(len(franja)):
            for j in range(i + 1, min(i + 7, len(franja))):
                d = distancia(franja[i], franja[j])
                if d < mejor_dist:
                    mejor_dist = d
                    mejor_par = (franja[i], franja[j])

        return mejor_par, mejor_dist

    return resolver(puntos_x)

#####
```

```
LISTA_3D = [(random.randrange(1,10000), random.randrange(1,10000), random.randrange(1,10000)) for x in range(1000)]
par, distancia = cercanos_3D_divide_y_vencerás(LISTA_3D)
print(f"Los dos puntos más cercanos son: {par} con distancia {distancia}")
```

Los dos puntos más cercanos son: ((2472, 5065, 8487), (2353, 5095, 8474)) con distancia 123.40988615179904

4.4.1 Complejidad del algoritmo de puntos más cercanos en 3D (Divide y Vencerás)

Sea $T(n)$ el número de operaciones necesarias para encontrar los dos puntos más cercanos en un conjunto de n puntos en el espacio tridimensional.

Caso base

- Para $n = 2$, se calcula una única distancia entre los dos puntos:
 $T(2) = 1$

Caso recursivo

Para $n > 2$, el algoritmo realiza:

1. Una división del conjunto de puntos ordenado por la coordenada x en dos mitades de tamaño aproximado $n/2$.
2. Una llamada recursiva para encontrar los puntos más cercanos en la mitad izquierda.
3. Una llamada recursiva para encontrar los puntos más cercanos en la mitad derecha.
4. Una fase de combinación que analiza los puntos cercanos a la frontera central, construyendo una franja de ancho igual a la mejor distancia encontrada y comparando solo un número constante de vecinos por cada punto de la franja.

Por tanto, la relación de recurrencia es:

$$T(n) = 2 \cdot T(n/2) + O(n)$$

donde el término $O(n)$ corresponde al coste de la fase de combinación de la franja razonando de la misma manera que en el caso 2D.

Resolución de la recurrencia

Aplicando lo mismo que en los casos anteriores a una recurrencia de la forma $a \cdot T(\frac{n}{b}) + c \cdot n^k$, tenemos que $a = 2$, $b = 2$, $k = 0$

Por tanto:

$$T(n) = O(n \log n)$$

Complejidad asintótica

El algoritmo requiere ordenar previamente los puntos por la coordenada x , lo que también tiene coste $O(n \log n)$.

Sumando ambos factores, la complejidad total del algoritmo es:

$O(n \log n)$